

OS x GPU Vendor Araç Haritası

Fine-tuning ve serving ortamı, işletim sistemine ve ekran kartı vendor'ına göre farklı araç zincirleri gerektiriyor. Bu sayfa TOOLING_MATRIX.md dosyasının karar mantığını ve tam matrisini görselleştiriyor.

KARAR AĞACI

1

GPU vendor ne?

Fine-tuning framework'ünü belirler — CUDA'ya bağımlı mı (Unsloth, Axolotl), ROCm'e mi, yoksa vendor-agnostic/Metal'e mi ihtiyaç var (MLX).

2

Bellek mimarisi dedicated mi, unified mi?

Dedicated VRAM'de (NVIDIA/AMD) kart sınıfı sabit bir tavan çizer. Unified memory'de (Apple Silicon) tavan toplam sistem RAM'idir ama sonradan yükseltilemez — donanım kararı burada geri döndürülemez.

3

Serving hedefi tekli kullanıcı mı, production mu?

`llama.cpp` hafif ve tek kullanıcı senaryosuna uygun; `vLLM` / `TGI` batch inference ve yüksek throughput için tasarlandı.

4

Quant formatı serving aracıyla uyumlu mu?

`GGUF` → `llama.cpp` ekosistemi; `AWQ` / `GPTQ` → `vLLM` / `TGI` ile daha native entegre. Yanlış eşleşme ekstra dönüştürme adımı doğurur (bu projede Qwen3 + MLX GGUF exporter'ında yaşandığı gibi).

MATRIS

	LINUX	WINDOWS	MACOS
NVIDIA CUDA · dedicated VRAM	ARAŞTIRMA Unsloth, Axolotl, LLaMA-Factory + vLLM / TGI / llama.cpp serving GGUF · AWQ · GPTQ	ARAŞTIRMA Unsloth (WSL2 önerilir), LLaMA-Factory + llama.cpp / vLLM (WSL2) GGUF · AWQ	– donanımsal olarak yok
AMD ROCm · dedicated VRAM	ARAŞTIRMA Axolotl (ROCm backend), LLaMA-Factory (kısmi) + llama.cpp (HIP) / vLLM (ROCm) GGUF	ARAŞTIRMA Sınırlı — çoğunlukla WSL2 üzerinden Linux yoluna düşer GGUF	– donanımsal olarak yok
Apple Silicon Metal/MLX · unified memory	– donanımsal olarak yok	– donanımsal olarak yok	DOĞRULANDI · BU PROJE MLX (mlx-lm) fine-tuning + llama.cpp (Metal backend) servis GGUF → Q4_K_M
CPU-only Intel iGPU · yalnız inference	ARAŞTIRMA llama.cpp (CPU) — fine-tuning teorik mümkün (Axolotl) ama pratik değil GGUF	ARAŞTIRMA llama.cpp (CPU) GGUF	ARAŞTIRMA llama.cpp (CPU, Metal olmadan) GGUF

Doğrulandı — elle test edildi

Araştırma aşaması — ekosistem bilgisine dayanıyor

Donanımsal olarak mevcut değil

ARAÇ SEÇİM NOTLARI — AYNI HÜCREDE BIRDEN FAZLA SEÇENEK VARSA

Unsloth vs Axolotl vs LLaMA-Factory LINUX/WINDOWS + NVIDIA

Unsloth — tek GPU'da hız ve düşük VRAM önceliğiye ilk tercih. Custom Triton kernel'leri ile 2x'e kadar hız, %50–80 daha az VRAM iddiası var; çoklu-GPU dağıtık eğitimde zayıf.

Axolotl — çoklu-GPU / DeepSpeed / FSDP gerekiyorsa ya da çok sayıda farklı model mimarisini tek YAML config akışında yönetmek istiyorsan.

LLaMA-Factory — kod yazmadan GUI üzerinden hızlı deneme/prototipleme için.

Bu proje bağlamı: tek GPU + tek model senaryosunda Unsloth ilk tercih olurdu. MLX'e geçiş yalnızca Apple Silicon'da CUDA/Triton olmadığı için oldu — Unsloth'un kendisi CUDA'da hâlâ en hızlı seçeneklerden biri, **terk edilmedi, sadece bu donanımda kullanılamadı**.

vLLM vs TGI vs llama.cpp SERVIS, NVIDIA/AMD

vLLM — çok kullanıcı, yüksek istek hacmi olan production servisi. PagedAttention ile batch throughput'ta öne çıkıyor.

TGI — Hugging Face ekosistemine (Inference Endpoints, Transformers uyumu) sıkı bağlıysan.

llama.cpp — tek kullanıcı / düşük trafik, hafif footprint, kolay dağıtım. GGUF'un taşınabilirliği de artı — bu projenin serving zincirinde de bu yüzden tercih edildi.

Neden Unsloth, Linux + AMD hücresinde yok ROCm SINIRI

Unsloth'un çekirdek hızlandırmaları CUDA/Triton kernel'lerine yazılı; resmi ROCm desteği yok — bu proje sırasında Apple Silicon/Metal'de de aynı sebeple çalışmadığı görüldü. Topluluk fork'ları ROCm'e taşımayı deniyor ama stabil/resmi değil. ROCm'de production için **Axolotl**'ün ROCm backend'ine güvenmek daha güvenli.

WSL2 vs native Windows WINDOWS + NVIDIA

Sadece serving/inference (örn. **llama-server**) çalıştırılacaksa native Windows yeterli, ekstra karmaşıklık katmıyor. Fine-tuning yapılacaksa WSL2'ye geçmek zaman kazandırıyor — Unsloth/Axolotl'ün Linux'ta test edilen kurulum adımları native Windows'a göre çok daha az sürtünmeli.

UNIFIED MEMORY — APPLE SILICON'DA "VRAM" YOK

Neden bu ayrım önemli

NVIDIA / AMD — Dedicated VRAM

GPU'nun CPU'dan ayrı, kendine ait belleği var (örn. RTX 4090 → 24GB). Model VRAM'e sığmazsa offload/swap gerekir. Büyütmek istersen kart değiştirirsin.

Apple Silicon — Unified Memory

CPU ve GPU aynı fiziksel RAM havuzunu paylaşır. Sığdırma sınırı toplam sistem RAM'i — ayrı bir GPU darboğazı yok, ama CPU ile bellek bant genişliği için rekabet var.

Geri döndürülemez karar: Apple Silicon'da RAM lehim (soldered) — sonradan yükseltilemez. Mac alırken seçilen RAM miktarı, GPU belleğinin de nihai tavanı oluyor. NVIDIA/AMD'de VRAM yetmezse kart değiştirilir; Apple'da bu yol yok.

Bu proje örneği: Josiefied-Qwen3-4B-abliterated-v1-4bit (2.26GB) 16GB'lık bir Mac'te rahat çalışıyor. Quantization (4-bit / Q4_K_M) unified memory'de de kritik kalıyor — kısıt artık "VRAM'e sığdırma" değil, RAM'i işletim sistemi ve diğer uygulamalarla paylaşabilme.

tooling_matrix.md — docs/mlx-finetune-test